task1.py | task1_output.txt | task2.py | task2_output.txt | task3.py | tas

```python
import os
import sys
import time
import numpy as np
from concurrent.futures import ThreadPoolExecutor
SIZE = 2_000_000
def serial_scan(array):  1 usage
    output = np.empty(len(array), dtype=np.int64)
    output[0] = array[0]
    for i in range(1, len(array)):
        output[i] = output[i - 1] + array[i]
    return output
def local_scan(array):  1 usage
    local_output = np.cumsum(array, dtype=np.int64)
    local_sum = int(local_output[-1])
    return local_output, local_sum
def apply_offset(data):  1 usage
    chunk, offset = data
    if offset != 0:
        chunk += offset
    return chunk
def parallel_scan(array, workers):  1 usage
    chunks = np.array_split(array, workers)

    with ThreadPoolExecutor(max_workers=workers) as executor:
        local_results = list(executor.map(local_scan, *iterables: chunks))

    local_outputs = [result[0] for result in local_results]
    chunk_sums = [result[1] for result in local_results]

    offsets = []
```

task1.py | task1_output.txt | task2.py | task2_output.txt | task3.py | task3_out

```
22      def parallel_scan(array, workers):  1 usage
29          chunk_sums = [result[1] for result in local_results]
30
31          offsets = []
32          running_sum = 0
33          for chunk_sum in chunk_sums:
34              offsets.append(running_sum)
35              running_sum += chunk_sum
36          offset_data = list(zip(local_outputs, offsets))
37          with ThreadPoolExecutor(max_workers=workers) as executor:
38              adjusted_chunks = list(executor.map(apply_offset, *iterables: offset_data))
39          return np.concatenate(adjusted_chunks)
40      def main():  1 usage
41          raw_id = input("Enter your numeric Student ID: ").strip()
42          try:
43              student_id = int(raw_id)
44              if student_id < 0:
45                  raise ValueError
46          except ValueError:
47              print("Student ID must be a non-negative integer.")
48              sys.exit(1)
49          rng = np.random.default_rng(student_id)
50          array = rng.integers(1, 11, size=SIZE, dtype=np.int64)
51          try:
52              import psutil
53              workers = psutil.cpu_count(logical=False) or 1
54          except ImportError:
55              workers = os.cpu_count() or 1
56          workers = max(1, min(workers, SIZE))
57          print("\nTASK 3 — Prefix Scan")
58          print(f"Student ID: {student_id}")
```

```
40      def main():  1 usage
56          workers = max(1, min(workers, SIZE))
57          print("\nTASK 3 — Prefix Scan")
58          print(f"Student ID: {student_id}")
59          print(f"Array size: {SIZE:,}")
60          print(f"Parallel workers: {workers}")
61
62          start = time.perf_counter()
63          serial_output = serial_scan(array)
64          serial_time = (time.perf_counter() - start) * 1000
65          start = time.perf_counter()
66          parallel_output = parallel_scan(array, workers)
67          parallel_time = (time.perf_counter() - start) * 1000
68          speedup = (
69              serial_time / parallel_time
70              if parallel_time > 0
71              else float("inf")
72          )
73          output_lines = [
74              "TASK 3 — Prefix Scan",
75              f"Student ID: {student_id}",
76              f"Array size: {SIZE}",
77              f"Parallel workers: {workers}",
78              f"Serial Time: {serial_time:.2f} ms",
79              f"Parallel Time: {parallel_time:.2f} ms",
80              f"Speedup: {speedup:.2f}x",
81          ]
82
83          try:
84              assert parallel_output[-1] == serial_output[-1]
85              output_lines.append("Verification Status: PASS")
```

```
82
83          try:
84              assert parallel_output[-1] == serial_output[-1]
85              output_lines.append("Verification Status: PASS")
86          except AssertionError:
87              output_lines.append("Verification Status: FAIL")
88              with open("task3_output.txt", "w", encoding="utf-8") as file:
89                  file.write("\n".join(output_lines) + "\n")
90              print("\n".join(output_lines))
91              sys.exit(1)
92
93          with open("task3_output.txt", "w", encoding="utf-8") as file:
94              file.write("\n".join(output_lines) + "\n")
95
96          print()
97          print("\n".join(output_lines))
98
99
100 if __name__ == "__main__":
101     main()
102
```